Profiling Matrix Multiplication

작성자: 김태용

목차

I. 실행환경
II. Naive Implement
III. Loop Reordering
IV. Loop Unrolling
V. Blocking
VI. SIMD
VII. 복합적 이용
VIII. 결론

I. 실행환경

Server: came03 (T350, 하-1310, 165.246.44.239)

서버에서 sudo dmidecode -t processor/memory, lscpu 명령어를 통해 확인하였다.

CPU: Intel Xeon E-2388G, 8C(3.2GHz, up to 5.1GHz) 16T

- L1d: 코어당 48 KB, 총 384 KB
- L1i: 코어당 32 KB, 총 256 KB
- L2: 코어당 512 KB, 총 4 MB
- L3: 16 MB
- L1과 L2 cache는 8-way associative[1] cache 구조이다.

Memory: 16GB * 4 = 64GB, 3200MT/s

해당 CPU의 캐시 구조는 다음 그림과 같다.

[1] 3.3.2 IA Core Level 1 and Level 2 Caches, “Intel® Xeon® E-2300 Processor Product Family Datasheet”, Intel 2021.

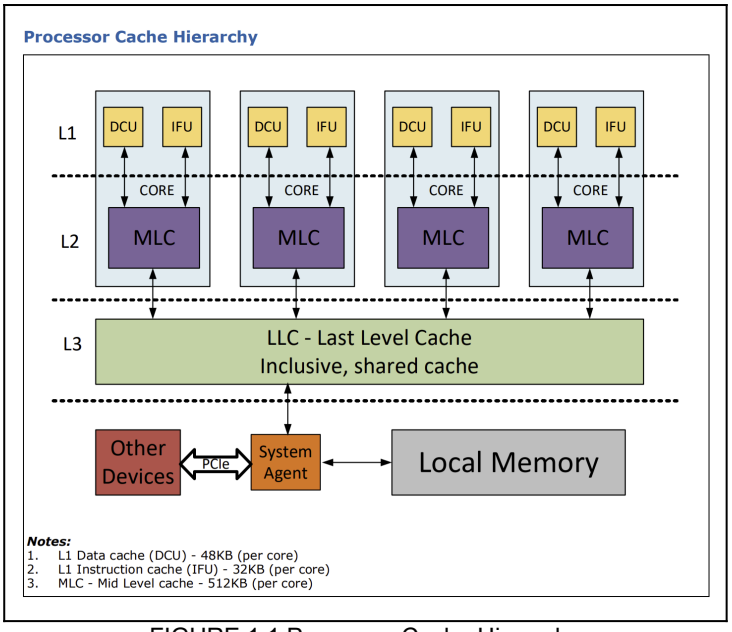

FIGURE 1.1 Processor Cache Hierarchy

Software: Intel VTune Profiler 2025.0.1 version

- Used Microarchitecture Exploration; CPU sampling interval: 0.1ms

Matrix: 1024*1024 2D int array

II. Naive Implement

일반적인 행렬곱의 정의대로 연산을 수행하는 구현이다. 앞으로 최적화 방법을 적용해 개선된 코드의 성능 향상 정도를 나타낼 때 해당 구현과 Naive 코드의 실행 cycle 수를 각각 비교하여 얼마나 개선되었는지 확인할 수 있다.

```
for (int i = 0; i < size; i++) {
    for (int j = 0; j < size; j++) {
        for (int k = 0; k < size; k++) {
            C[i][j] += A[i][k] * B[k][j];
        }
    }
}
```

FIGURE 2.1 Naive Implement

III. Loop reordering

FIGURE 2.1에서 볼 수 있는 것처럼 일반적으로 행렬곱 연산을 수행할 때 A행렬의 행과 B행렬의 열을 곱하고, i-j-k 순서로 반복문이 진행된다. 컴퓨터에서는 cache miss에 대해 블록 단위로 데이터를 가져오는데, 만약 어떤 행렬의 행 길이가 블록의 크기보다 크다면 그 다음 행의 정보는 캐시에 저장되지 않으므로 열 단위로 실행되며 FIGURE 3.1와 같이 지속적으로 miss가 발생할 가능성이 있다.

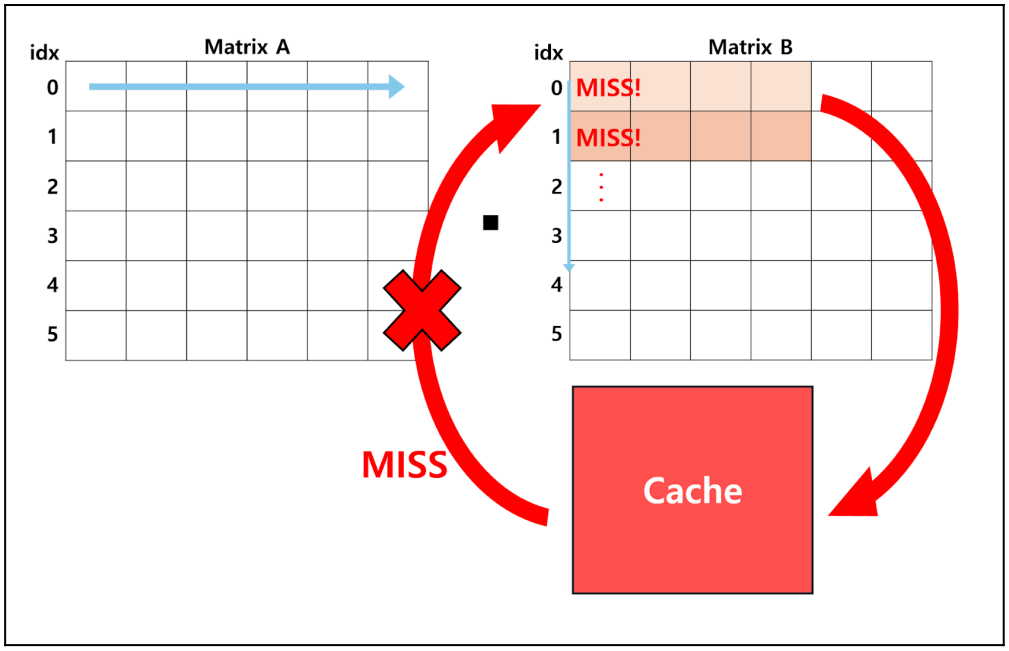

FIGURE 3.1 Example of cache miss incurring by order of loop

즉, ‘c[i][j] = c[i][j] + a[i][k] * b[k][j];’의 배열 접근 순서에 맞게 중첩 반복문의 수행 순서를 i-k-j로 바꾸면 배열에서 데이터가 배치된 순서대로 접근하기 때문에 공간 지역성을 바탕으로 성능이 향상될 것이다.

1. 실행 결과

Naive Execution cycles	= 10589426291
Optimized Execution cycles	= 9533227892
Performance Improvement	= 1.110791

FIGURE 3.1 Loop reordering Execution cycles

2. 결과 분석

```
for (int i = 0; i < size; i++) {
    for (int k = 0; k < size; k++) {
        for (int j = 0; j < size; j++) {
            C[i][j] += A[i][k] * B[k][j];
        }
    }
}
```

FIGURE 3.2 Loop reordering Implement

실제로 실행해본 결과를 비교하면 11% 내외의 성능 향상이 있었는데, 기대와는 다르게 Naive 구현과 대단한 차이를 보이지는 않는다. 이를 통해 원래 loop의 순서를 변경하기 전에도 Matrix B가 Cache에 일정 부분 hit되고 있었고, 행렬의 크기가 커질수록 성능 향상 비율이 커질 것이라고 추론할 수 있다. 예측의 타당성을 확인해보자.

3. 추가 실험

위 FIGURE 3.1에서도 확인할 수 있듯, 행렬의 크기에 비해 캐시가 충분히 큰 상황이라면 Loop reordering의 효과가 미미할 것이다. 행렬의 크기를 32*32부터 5120*5120까지 변화시키며 Loop reordering의 성능 향상 비율을 비교하는 실험을 통해 예측을 확인하겠다.

실행시 오차가 존재할 수 있으므로 각 3회 실행한 결과에 대해 평균을 내어 비교하고, 이를 그래프로 나타내면 다음과 같다.

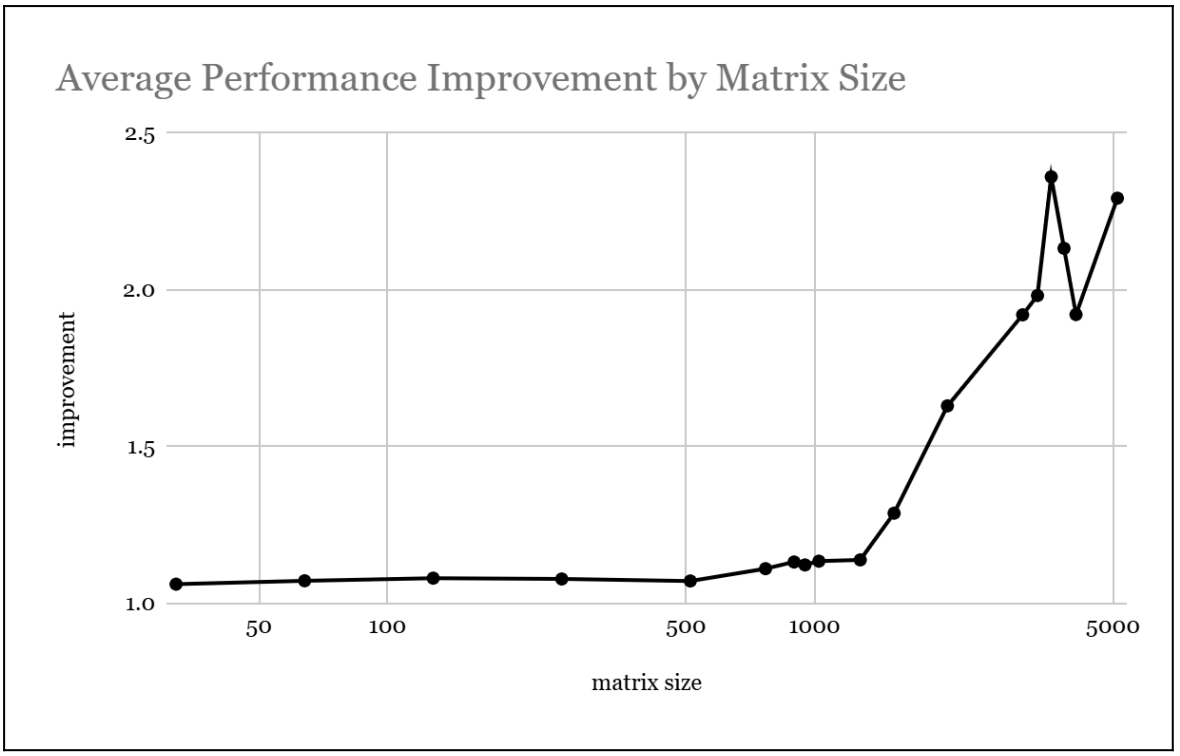

FIGURE 3.3 Average Performance Improvement by Matrix Size

예상한 바와 같이 성능 개선치는 행렬 크기에 비례하는 모습을 보인다. 행렬 크기(~512)의 상대적으로 작은 행렬에서는 행렬이 캐시에 잘 들어가 있어 루프 순서를 바꾸지 않더라도 cache hit이 잘 발생하는 것을 VTune을 통해 확인하였다. 이로 인해 성능 개선치가 크지 않으나 행렬의 크기가 커질수록 naive한 GEMM에 비해 루프 순서를 변경했을 때 캐시 활용을 훨씬 잘 하므로 성능이 좋아지는 것을 확인할 수 있다.

유의미한 그래프가 맞는지에 대해서는 의문이 든다. 약 370*370 부분에서 L2 캐시 크기를 벗어나고, 2000*2000 부분부터 L3 캐시 크기를 벗어나는데 이 때, Naive GEMM의 캐시 접근 구조가 더 비효율적이고 miss 시점이 더 이를 것이므로, 또 cache의 한 단계를 벗어날수록 접근에 필요한 시간이 기하급수적으로 늘어나므로 어디까지나 Naive GEMM이 많이 느려짐으로 인한 성능 개선 수치라고도 생각한다. FIGURE 3.3을 통해 명확하게 확인할 수 있는 것은 상대적으로 작은 행렬에 대해 루프 순서를 바꾸지 않더라도 cache hit이 잘 발생할 것이라는 점이다.

IV. Loop unrolling

Instruction Scheduling시 dependency가 있는 명령어들이 없는 명령어에 비해 상대적으로 많으면 개선할 수 있는 양이 한정된다. 명령어의 개수를 늘려 이를 극복할 수 있는 방법이 Loop Unrolling이다. loop를 풀면 여러 번 반복할 iteration을 하나의 코드로 보아 한 번에 수행할 명령어의 개수를 늘릴 수 있고, 명령어 간 dependency의 영향력을 분산시킬 수 있다.

Pros	**Constraints**
● 서로 다른 register를 사용하게 하는 register renaming으로 dependency 해소 ● 증감, branch 등 Loop body 외 부분의 비중 감소	● Loop의 크기를 대략적으로 알 필요 ● Code의 길이가 과도하게 길어지면 캐시 범위를 벗어나는 등 문제 발생 가능

FIGURE 2.5 Pros and Constraints of Loop Unrolling

Loop unrolling을 할 때 행렬곱의 세 loop인 i, j, k에 대해 각각 몇 개의 iteration을 하나의 코드 로 만들지 직접 지정해주어야 한다. 어떤 loop에 대해 몇 개를 풀어주는 것이 성능 향상에 도움이 될지 실험을 통해 알아보겠다.

우선 i, j, k에 대해 단독으로 수행하였다. 각각 다른 변수는 1로 고정해두고 값을 올려보며 성능 향상 여부를 확인해 보았다. 실행 시 오차가 발생할 수 있으므로 각 5회 실행한 결과를 평균 내어 비교하였다.

1. 실행 결과

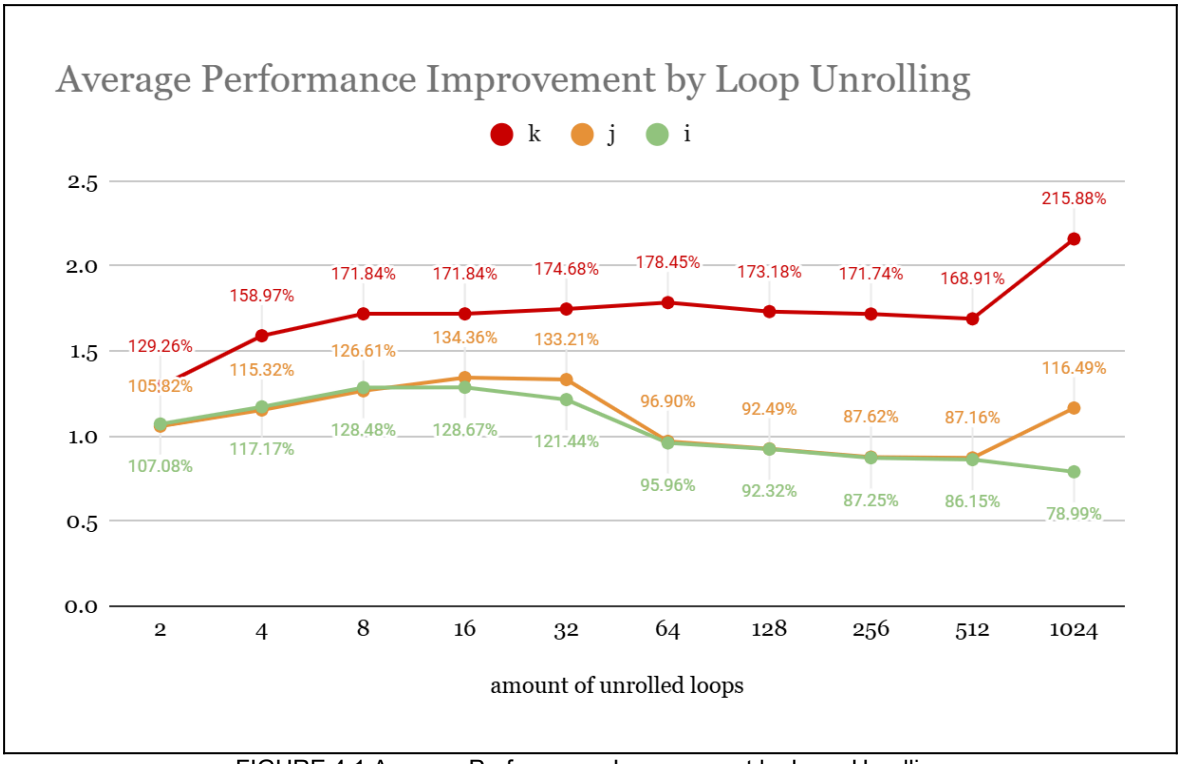

FIGURE 4.1 Average Performance Improvement by Loop Unrolling

2. 결과 분석

우선 해당 실험은 loop reordering이 적용되지 않은 코드를 실행한 것이므로 loop의 실행 순서가 i-j-k이다. 지속적으로 k에 대해 먼저 접근하고 있고, 이로 인해 k loop가 풀어졌을 때 가장 좋은 효과를 보인다. 또한, k가 64, j와 i는 16을 기점으로 성능 개선 비율이 꺾이는 부분은 해당 시점 이후로 명령어 의존성 개선에 의한 성능 향상보다 register spilling 현상, 캐시 범위를 벗어나는 등의 문제로 성능이 저하되는 것이 더 크다고 볼 수 있겠다.

특이한 지점은 1024에 k, j의 성능이 상당히 개선되고 i는 그렇지 않다는 점인데, 이는 loop가 완전히 사라짐으로써 branch, jump 등 명령어가 사라져 개선되는 것으로 보인다. 중첩 반복문의 가장 내부에 위치한 k의 성능 향상 폭이 가장 크고, i는 어차피 가장 외부에 있으므로 성능이 향상되지 않았다는 사실 역시 이를 뒷받침한다. VTune을 통해 기존 어셈블리와 비교하여 브랜치가 차지하는 비중을 확인하였고, 가정이 옳음을 알 수 있었다.

V. Blocking

Locality를 아무리 잘 고려하더라도 접근할 데이터의 크기가 너무 크다면 캐시의 크기를 넘어서기 때문에 iteration마다 miss가 발생할 수밖에 없다. 캐시의 크기만 충분히 컸더라면 발생하지 않았을 miss이기도 하다. 이에 대한 해결 방법이 Blocking이다. 본래 한 번에 수행하여 miss가 발생했던 일을 캐시에 hit될 수 있도록 더 작은 block 단위로 쪼개어 수행하는 방법이다.

FIGURE 2.5를 보자, 원래 행렬의 크기가 캐시보다 더 컸으나 Blocking을 이용해 작은 block들로 분할하면 cache에 잘 들어갈 수 있음을 보여준다.

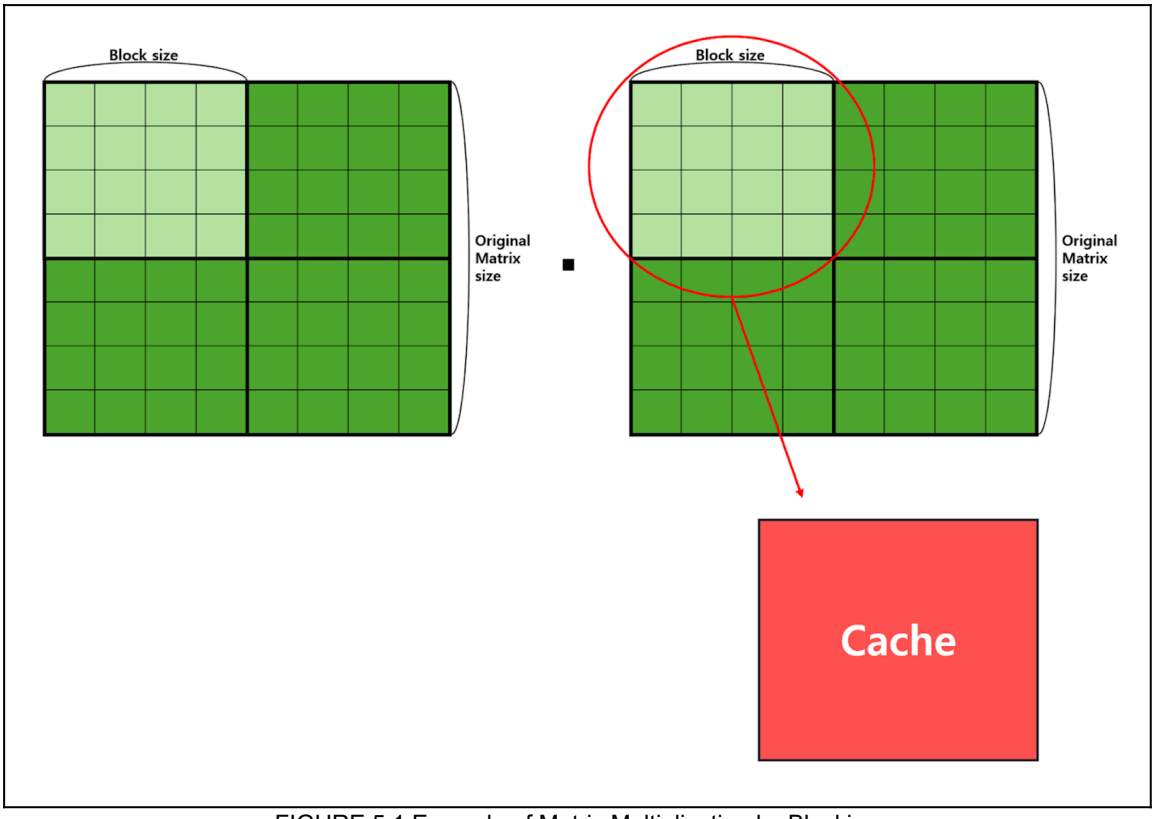

FIGURE 5.1 Example of Matrix Multiplication by Blocking

이 때, 우리가 고려할 cache는 단순히 한 개만 존재하는 것이 아니라 L1, L2, L3로 여러 개 존재한다. 행렬 크기에 맞는 적절한 block의 크기를 사용하는 것이 blocking에서 중요할 것이므로 block의 크기를 2부터 1024까지 2배씩 늘려 가며 Naive 구현에 비해 해당 block size를 사용하는 blocking을 적용하면 성능 개선이 얼마나 일어나는지 비교해 보겠다.

1. 실행 결과

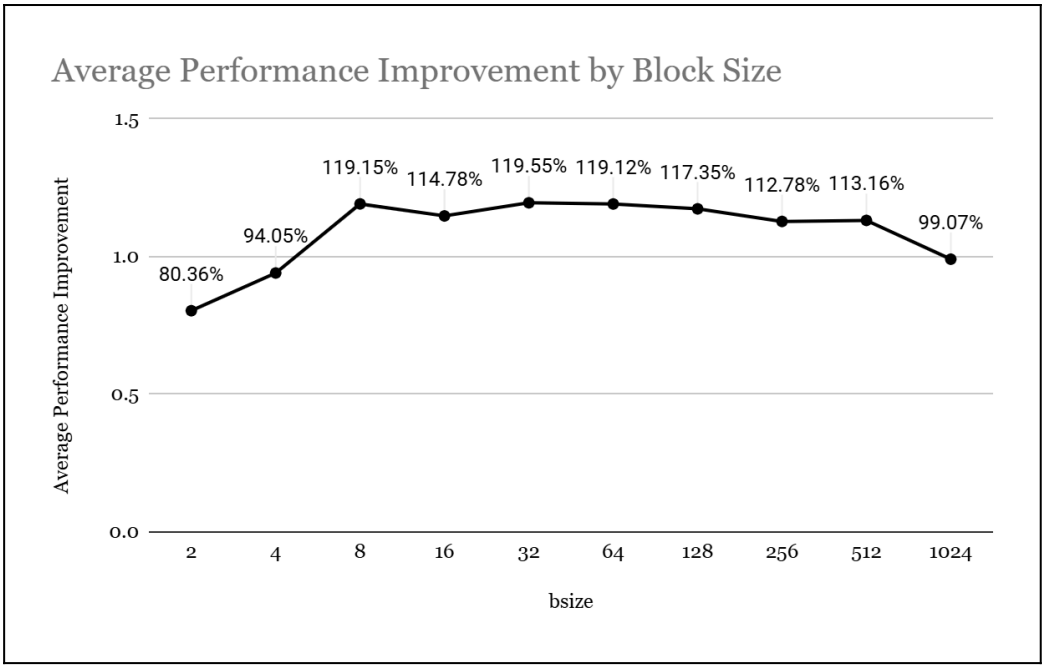

FIGURE 5.2 Average Performance Improvement for 1024*1024 matrix by Block Size

2. 결과 분석

실행마다 편차가 존재하기 때문에 각 block size별로 3회씩 실행한 개선 비율을 평균 내어 그래프로 만들었다. block size 8, 32, 64가 약 119%로 가장 높은 개선율을 보였다.

bsize=128의 경우에는 128^2*4/1000KB, 즉 한 블록당 약 66KB를 사용한다. 따라서 L1 data 캐시의 크기인 48KB를 넘는 크기이고, 256, 512의 경우에는 L2 캐시 범위까지 넘어가므로 block size가 커질수록 성능이 낮아지는 결과를 보인다.

bsize=8 ~ 64은 L1 data 캐시 안에서 실행될 수 있기 때문에 가장 좋은 성능을 보이는 것을 확인할 수 있다. bsize가 8보다 더 작을 때에는 오히려 loop 연산이 추가되며 branch 등을 더 수행하는 overhead가 더 커 성능이 오히려 떨어진다.

VI. SIMD

SIMD는 Single Instruction Multi Data의 약자로, 하나의 명령어를 통해 여러 가지의 데이터를 동시에 연산할 수 있는 명령어 집합을 말한다. CPU에서 제공하는 SIMD 명령어를 활용하여 여러 데이터를 동시에 연산할 수 있고, 이를 통해 연산 속도를 높일 수 있다.

SIMD 사용 시 하드웨어에서 지원하여야 한다. 확인해본 결과 came03 서버의 cpu인 E-2388v에서는 SSE, SSE2, SSE3, SSE4_1, SSE4_2, AVX, AVX2, AVX512를 지원하고 SSE4a와 SSE5는 지원하지 않는다.

1. 실행 결과

5회 실행한 cycle 합에 대한 평균 개선율
Naive Execution cycles = 11055250282
Optimized Execution cycles = 1712144170

Performance Improvement = 6.456963

FIGURE 6.1 SIMD Execution cycles

2. 결과 분석

```
for (int i = 0; i < size; i++){
    for (int k = 0; k < size; k++){
        __m512i A_vec = _mm512_set1_epi64(A[i][k]);
        for (int j = 0; j< size; j+=16){
            __m512i B_vec = _mm512_loadu_si512((__m512i*) &B[k][j]);
            __m512i C_vec = _mm512_loadu_si512((__m512i*) &C[i][j]);
            C_vec = _mm512_add_epi64(_mm512_mullo_epi64(A_vec, B_vec), C_vec);
            _mm512_storeu_si512((__m512i*) C[i][j], C_vec);
        }
    }
}
```

FIGURE 6.2 GEMM with SIMD

512비트 연산을 한번에 수행할 수 있는 AVX512를 이용하여 GEMM을 구현한 코드이다. 현재 32비트인 int 타입의 행렬에 대해 연산을 수행하고 있으므로 16개의 정수 덧셈 연산을 한번에 수행할 수 있다. 실제로 성능 개선율 또한 약 6.5배로 높은 것을 확인할 수 있다.

VII. 복합적 이용

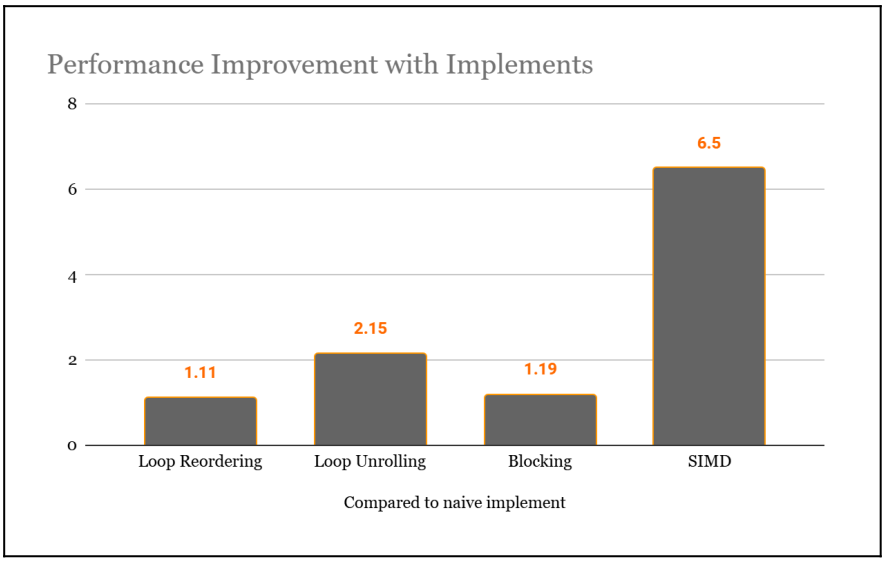

FIGURE 7.1 Performance Improvement with Implements

FIGURE 7.1은 여러 GEMM 최적화 방법들을 적용했을 때의 naive 대비 성능 개선율을 도식화한 것이다. 해당 최적화 방법들을 복합적으로 적용했을 때 과연 성능 향상이 얼마나 이루어질지, 또 개선 폭은 단독으로 적용했을 때와 비교하면 얼마나 클지 확인해보자.

a. Blocking + Unrolling

단독으로 적용했을 때 가장 좋은 성능을 보였던 bsize = 64의 blocking과 loop를 완전히 해체하는 unrolling을 적용해보았다. block별로 unroll해줘야 하므로 loop를 해체하기 위해서는 k에 대해 64개를 풀어주면 된다.

5회 실행한 cycle 합에 대한 평균 개선율
Naive Execution cycles = 54648492496
Optimized Execution cycles = 23745525903
Performance Improvement = 2.301423

FIGURE 7.2 Blocking(64) + Unrolling Execution cycles

원래 unrolling 수행 시에는 loop를 완전히 해체하기 위해서 k에 대해 1024개를 풀어주었다. 이 때 한 번에 수행하는 코드 라인 수가 너무 많아 일어나는 병목이 존재하나 branch가 발생하지 않아 얻는 이득이 더 커 성능 개선이 일어나는 상황이었다. Blocking을 적용한 뒤 unroll한다면 branch가 발생하지 않아 얻는 이득은 그대로 존재하면서 한 번에 수행할 코드 라인 수는 더 적어져 성능이 더욱 개선될 수 있을 것이라 예상하였다.

실제로 실행해보면 수치가 원래 예상할 수 있었던 2.15*1.19 = 2.559에 근접한 것을 확인할 수 있다. VTune을 통해 확인해보면 원래 unrolling code에서는 배열 주소 접근의 [k] 부분을 상수로 직접 접근하고 있었는데, blocking 내부에서 unroll하면 해당 블록의 시작 위치(sk)에 상수를 더해줘 주소에 접근하고 있다. 이 부분에서 원래 발생하지 않았던 성능 저하가 존재한다.

확인을 위해 해당 부분과 correction check 부분을 제외하고 unrolling code와 같이 [k]로 실행해 보니 예상 수치를 넘어서는 약 3배의 성능 개선을 확인할 수 있었다.

j까지 unroll하는 것과 block size 변경을 진행해보았으나 성능이 개선되지 않아 생략하겠다.

b. SIMD + Blocking + Reordering + Unrolling

Naive Execution cycles = 11182945459
Optimized Execution cycles = 1219438705
Performance Improvement = 9.170568

FIGURE 7.3 SIMD + Blocking(64) + Reordering + Unrolling(1/2/1)

Naive Execution cycles = 59068273969
Optimized Execution cycles = 5643000363
Performance Improvement = 10.467529

FIGURE 7.4 SIMD + Blocking(32) + Redordering + Unrolling(1/2/1)

Naive Execution cycles = 67066992620
Optimized Execution cycles = 4642374796
Performance Improvement = 14.446699

FIGURE 7.5 SIMD + Blocking(64) + Reordering + Unrolling(1/64/1)

Naive Execution cycles = 67066992620
Optimized Execution cycles = 4642374796
Performance Improvement = 14.446699

FIGURE 7.6 SIMD + Blocking(64) + Reordering + Unrolling(1/64/1)

Naive Execution cycles = 62805026590
Optimized Execution cycles = 4223736357
Performance Improvement = 14.869542
bsize 128, 2/128/1

FIGURE 7.7 SIMD + Blocking(128) + Reordering + Unrolling(2/128/1)

Naive Execution cycles = 64582149805
Optimized Execution cycles = 4247340690
Performance Improvement = 15.205314

FIGURE 7.8 SIMD + Blocking(128) + Reordering + Unrolling(2/128/2)

FIGURE 7.3-8의 여러 방식으로 SIMD에 기존 최적화 방법들을 결합해 보았다. 확인해 본 것 중 가장 높은 성능 개선은 FIGURE 7.8의 15배였다. 구현은 다음과 같다.

```
for (int si = 0; si<size; si+= bsize){
        for (int sk = 0; sk < size; sk += bsize){
            for (int sj = 0; sj < size; sj+= bsize){

                for (int i = si; i < si + bsize; i+=2){
                    __m512i C_vec_1 = _mm512_loadu_epi32((__m512i*) &C[i][sj]);
                    __m512i C_vec_2 = _mm512_loadu_epi32((__m512i*) &C[i][sj+16]);
                    __m512i C_vec_3 = _mm512_loadu_epi32((__m512i*) &C[i][sj+32]);
                    __m512i C_vec_4 = _mm512_loadu_epi32((__m512i*) &C[i][sj+48]);
                    __m512i C_vec_5 = _mm512_loadu_epi32((__m512i*) &C[i][sj+64]);
                    __m512i C_vec_6 = _mm512_loadu_epi32((__m512i*) &C[i][sj+80]);
                    __m512i C_vec_7 = _mm512_loadu_epi32((__m512i*) &C[i][sj+96]);
                    __m512i C_vec_8 = _mm512_loadu_epi32((__m512i*) &C[i][sj+112]);

                    __m512i C_vec_9 = _mm512_loadu_epi32((__m512i*) &C[i+1][sj]);
                    __m512i C_vec_10 = _mm512_loadu_epi32((__m512i*) &C[i+2][sj+16]);
                    __m512i C_vec_11 = _mm512_loadu_epi32((__m512i*) &C[i+3][sj+32]);
                    __m512i C_vec_12 = _mm512_loadu_epi32((__m512i*) &C[i+4][sj+48]);
                    __m512i C_vec_13 = _mm512_loadu_epi32((__m512i*) &C[i+5][sj+64]);
                    __m512i C_vec_14 = _mm512_loadu_epi32((__m512i*) &C[i+6][sj+80]);
                    __m512i C_vec_15= _mm512_loadu_epi32((__m512i*) &C[i+7][sj+96]);
                    __m512i C_vec_16 = _mm512_loadu_epi32((__m512i*) &C[i+8][sj+112]);
                    for (int k = sk; k < sk + bsize; k+=2){
                        __m512i A_vec = _mm512_set1_epi32(A[i][k]);

                        __m512i B_vec_1 = _mm512_loadu_epi32((__m512i*) &B[k][sj]);
                        __m512i B_vec_2 = _mm512_loadu_epi32((__m512i*) &B[k][sj+16]);
```

```
__m512i B_vec_3 = _mm512_loadu_epi32((__m512i*) &B[k][sj+32]);
__m512i B_vec_4 = _mm512_loadu_epi32((__m512i*) &B[k][sj+48]);
__m512i B_vec_5 = _mm512_loadu_epi32((__m512i*) &B[k][sj+64]);
__m512i B_vec_6 = _mm512_loadu_epi32((__m512i*) &B[k][sj+80]);
__m512i B_vec_7 = _mm512_loadu_epi32((__m512i*) &B[k][sj+96]);
__m512i B_vec_8 = _mm512_loadu_epi32((__m512i*) &B[k][sj+112]);

C_vec_1 = _mm512_add_epi32(C_vec_1, _mm512_mullo_epi32(A_vec, B_vec_1));
C_vec_2 = _mm512_add_epi32(C_vec_2, _mm512_mullo_epi32(A_vec, B_vec_2));
C_vec_3 = _mm512_add_epi32(C_vec_3, _mm512_mullo_epi32(A_vec, B_vec_3));
C_vec_4 = _mm512_add_epi32(C_vec_4, _mm512_mullo_epi32(A_vec, B_vec_4));
C_vec_5 = _mm512_add_epi32(C_vec_5, _mm512_mullo_epi32(A_vec, B_vec_5));
C_vec_6 = _mm512_add_epi32(C_vec_6, _mm512_mullo_epi32(A_vec, B_vec_6));
C_vec_7 = _mm512_add_epi32(C_vec_7, _mm512_mullo_epi32(A_vec, B_vec_7));
C_vec_8 = _mm512_add_epi32(C_vec_8, _mm512_mullo_epi32(A_vec, B_vec_8));

_mm512_storeu_epi32((__m512i*) &C[i][sj], C_vec_1);
_mm512_storeu_epi32((__m512i*) &C[i][sj+16], C_vec_2);
_mm512_storeu_epi32((__m512i*) &C[i][sj+32], C_vec_3);
_mm512_storeu_epi32((__m512i*) &C[i][sj+48], C_vec_4);
_mm512_storeu_epi32((__m512i*) &C[i][sj+64], C_vec_5);
_mm512_storeu_epi32((__m512i*) &C[i][sj+80], C_vec_6);
_mm512_storeu_epi32((__m512i*) &C[i][sj+96], C_vec_7);
_mm512_storeu_epi32((__m512i*) &C[i][sj+112], C_vec_8);

A_vec = _mm512_set1_epi32(A[i][k+1]);

B_vec_1 = _mm512_loadu_epi32((__m512i*) &B[k+1][sj]);
B_vec_2 = _mm512_loadu_epi32((__m512i*) &B[k+1][sj+16]);
B_vec_3 = _mm512_loadu_epi32((__m512i*) &B[k+1][sj+32]);
B_vec_4 = _mm512_loadu_epi32((__m512i*) &B[k+1][sj+48]);
B_vec_5 = _mm512_loadu_epi32((__m512i*) &B[k+1][sj+64]);
B_vec_6 = _mm512_loadu_epi32((__m512i*) &B[k+1][sj+80]);
B_vec_7 = _mm512_loadu_epi32((__m512i*) &B[k+1][sj+96]);
B_vec_8 = _mm512_loadu_epi32((__m512i*) &B[k+1][sj+112]);

C_vec_1 = _mm512_add_epi32(C_vec_1, _mm512_mullo_epi32(A_vec, B_vec_1));
C_vec_2 = _mm512_add_epi32(C_vec_2, _mm512_mullo_epi32(A_vec, B_vec_2));
C_vec_3 = _mm512_add_epi32(C_vec_3, _mm512_mullo_epi32(A_vec, B_vec_3));
C_vec_4 = _mm512_add_epi32(C_vec_4, _mm512_mullo_epi32(A_vec, B_vec_4));
C_vec_5 = _mm512_add_epi32(C_vec_5, _mm512_mullo_epi32(A_vec, B_vec_5));
C_vec_6 = _mm512_add_epi32(C_vec_6, _mm512_mullo_epi32(A_vec, B_vec_6));
C_vec_7 = _mm512_add_epi32(C_vec_7, _mm512_mullo_epi32(A_vec, B_vec_7));
C_vec_8 = _mm512_add_epi32(C_vec_8, _mm512_mullo_epi32(A_vec, B_vec_8));

_mm512_storeu_epi32((__m512i*) &C[i][sj], C_vec_1);
_mm512_storeu_epi32((__m512i*) &C[i][sj+16], C_vec_2);
_mm512_storeu_epi32((__m512i*) &C[i][sj+32], C_vec_3);
_mm512_storeu_epi32((__m512i*) &C[i][sj+48], C_vec_4);
_mm512_storeu_epi32((__m512i*) &C[i][sj+64], C_vec_5);
_mm512_storeu_epi32((__m512i*) &C[i][sj+80], C_vec_6);
_mm512_storeu_epi32((__m512i*) &C[i][sj+96], C_vec_7);
_mm512_storeu_epi32((__m512i*) &C[i][sj+112], C_vec_8);

A_vec = _mm512_set1_epi132(A[i+1][k+1]);

B_vec_1 = _mm512_loadu_epi32((__m512i*) &B[k+1][sj]);
B_vec_2 = _mm512_loadu_epi32((__m512i*) &B[k+1][sj+16]);
B_vec_3 = _mm512_loadu_epi32((__m512i*) &B[k+1][sj+32]);
B_vec_4 = _mm512_loadu_epi32((__m512i*) &B[k+1][sj+48]);
B_vec_5 = _mm512_loadu_epi32((__m512i*) &B[k+1][sj+64]);
B_vec_6 = _mm512_loadu_epi32((__m512i*) &B[k+1][sj+80]);
B_vec_7 = _mm512_loadu_epi32((__m512i*) &B[k+1][sj+96]);
B_vec_8 = _mm512_loadu_epi32((__m512i*) &B[k+1][sj+112]);

C_vec_1 = _mm512_add_epi32(C_vec_1, _mm512_mullo_epi32(A_vec, B_vec_1));
C_vec_2 = _mm512_add_epi32(C_vec_2, _mm512_mullo_epi32(A_vec, B_vec_2));
C_vec_3 = _mm512_add_epi32(C_vec_3, _mm512_mullo_epi32(A_vec, B_vec_3));
C_vec_4 = _mm512_add_epi32(C_vec_4, _mm512_mullo_epi32(A_vec, B_vec_4));
C_vec_5 = _mm512_add_epi32(C_vec_5, _mm512_mullo_epi32(A_vec, B_vec_5));
C_vec_6 = _mm512_add_epi32(C_vec_6, _mm512_mullo_epi32(A_vec, B_vec_6));
C_vec_7 = _mm512_add_epi32(C_vec_7, _mm512_mullo_epi32(A_vec, B_vec_7));
C_vec_8 = _mm512_add_epi32(C_vec_8, _mm512_mullo_epi32(A_vec, B_vec_8));

_mm512_storeu_epi32((__m512i*) &C[i+1][sj], C_vec_1);
_mm512_storeu_epi32((__m512i*) &C[i+1][sj+16], C_vec_2);
_mm512_storeu_epi32((__m512i*) &C[i+1][sj+32], C_vec_3);
_mm512_storeu_epi32((__m512i*) &C[i+1][sj+48], C_vec_4);
_mm512_storeu_epi32((__m512i*) &C[i+1][sj+64], C_vec_5);
_mm512_storeu_epi32((__m512i*) &C[i+1][sj+80], C_vec_6);
_mm512_storeu_epi32((__m512i*) &C[i+1][sj+96], C_vec_7);
_mm512_storeu_epi32((__m512i*) &C[i+1][sj+112], C_vec_8);

A_vec = _mm512_set1_epi32(A[i+1][k+1]);
```

```
                B_vec_1 = _mm512_loadu_epi32((__m512i*) &B[k+1][sj]);
                B_vec_2 = _mm512_loadu_epi32((__m512i*) &B[k+1][sj+16]);
                B_vec_3 = _mm512_loadu_epi32((__m512i*) &B[k+1][sj+32]);
                B_vec_4 = _mm512_loadu_epi32((__m512i*) &B[k+1][sj+48]);
                B_vec_5 = _mm512_loadu_epi32((__m512i*) &B[k+1][sj+64]);
                B_vec_6 = _mm512_loadu_epi32((__m512i*) &B[k+1][sj+80]);
                B_vec_7 = _mm512_loadu_epi32((__m512i*) &B[k+1][sj+96]);
                B_vec_8 = _mm512_loadu_epi32((__m512i*) &B[k+1][sj+112]);

                C_vec_1 = _mm512_add_epi32(C_vec_1, _mm512_mullo_epi32(A_vec, B_vec_1));
                C_vec_2 = _mm512_add_epi32(C_vec_2, _mm512_mullo_epi32(A_vec, B_vec_2));
                C_vec_3 = _mm512_add_epi32(C_vec_3, _mm512_mullo_epi32(A_vec, B_vec_3));
                C_vec_4 = _mm512_add_epi32(C_vec_4, _mm512_mullo_epi32(A_vec, B_vec_4));
                C_vec_5 = _mm512_add_epi32(C_vec_5, _mm512_mullo_epi32(A_vec, B_vec_5));
                C_vec_6 = _mm512_add_epi32(C_vec_6, _mm512_mullo_epi32(A_vec, B_vec_6));
                C_vec_7 = _mm512_add_epi32(C_vec_7, _mm512_mullo_epi32(A_vec, B_vec_7));
                C_vec_8 = _mm512_add_epi32(C_vec_8, _mm512_mullo_epi32(A_vec, B_vec_8));

                _mm512_storeu_epi32((__m512i*) &C[i+1][sj], C_vec_1);
                _mm512_storeu_epi32((__m512i*) &C[i+1][sj+16], C_vec_2);
                _mm512_storeu_epi32((__m512i*) &C[i+1][sj+32], C_vec_3);
                _mm512_storeu_epi32((__m512i*) &C[i+1][sj+48], C_vec_4);
                _mm512_storeu_epi32((__m512i*) &C[i+1][sj+64], C_vec_5);
                _mm512_storeu_epi32((__m512i*) &C[i+1][sj+80], C_vec_6);
                _mm512_storeu_epi32((__m512i*) &C[i+1][sj+96], C_vec_7);
                _mm512_storeu_epi32((__m512i*) &C[i+1][sj+112], C_vec_8);
            }
        }
    }
  }
}
```

FIGURE 7.5 GEMM with SIMD, Blocking(128), Unrolling(2/128/2), Reordering

VIII. 결론 및 느낀 점

Matrix Multiplication 최적화를 수행하고 이에 대한 보고서를 작성하며 하드웨어 구조를 잘 반영하는 소프트웨어를 설계하는 것이 중요하다고 다시금 느꼈다. 메모리 관련하여 부하가 발생할 수 있는 지점이 굉장히 많다는 생각이 들었고, roofline model에서 확인할 수 있는 것처럼 cpu의 연산량이 발전하더라도 메모리를 사용하는 방식을 최적화하지 못하면 결국 이를 제대로 활용하지 못한다는 부분을 실감할 수 있었다.

또한 특이하다는 생각이 드는 부분을 실제로 확인해보며 나아가 간단하게나마 실험을 설계하고 수행해 보는 과정이 생소하지만 새롭고 흥미로웠다. 다른 주제에 대해서도 탐구할 수 있는 기회가 있으면 도전해보고 싶다.